Memorando

Aplicações Móveis, 2026/2027

De: Emanuel Carneiro dos Santos
Nº de Matrícula: 20230429
Assunto: Introdução ao Android
Data: 22/10/2026

1. Introdução

O presente laboratório tem como principal objectivo a introdução ao desenvolvimento de aplicações Android, familiarizando o estudante com as duas ferramentas centrais do ecossistema: o **Android Studio**, ambiente integrado de desenvolvimento, e o **Gradle**, sistema de automação de construção.

O Android é uma pilha de software de código aberto baseada no kernel Linux, na qual as aplicações são construídas a partir de componentes: **Activities**, **Services**, **Broadcast Receivers** e **Content Providers**, que comunicam entre si através de mensagens designadas **Intents**. A interface de cada actividade é descrita em XML como uma árvore hierárquica de **Views** e **ViewGroups**, e o ficheiro `AndroidManifest.xml` declara o esqueleto da aplicação. Todo o processo de construção: compilação do código-fonte, processamento de recursos e empacotamento no APK, é orquestrado pelo Gradle, através do Android Gradle Plugin.

São três os exercícios abordados: o **Exercício I** implementa uma aplicação *Hello World* com duas actividades, percorrendo a criação do projecto, a execução num dispositivo, a depuração com o Logcat e a navegação entre actividades através de Intents; o **Exercício II** desmistifica o Gradle, desde as tarefas básicas e o plugin Java até aos projectos multi-módulo e à sua integração com o Android Studio; o **Exercício III**, tratado como desafio, consiste numa aplicação de lista de tarefas com uma única actividade.

Quanto às restrições, os exercícios de linha de comandos usaram a distribuição gradle-5.2.1 indicada no enunciado, o que implica uma JVM compatível (JDK 11) e a sintaxe legada de dependências (`compile`/`testCompile`). Os projectos Android foram criados com a versão actual do Android Studio (Rabbit 1 | 2026.2.1), em Java, com o SDK mínimo API 23, e testados num dispositivo real Samsung SM-A105G, em vez do emulador.

OBS: As respostas às perguntas do laboratório encontram-se no final de cada subsecção que contém perguntas.

2. Experiências Realizadas

Descrevem-se, de seguida, os Exercícios I e II do enunciado, pela ordem em que foram realizados. O Exercício III é apresentado na secção 3 (Desafios).

2.1. Exercício I – Hello World

O objectivo deste exercício é implementar uma aplicação com duas actividades: a primeira recolhe uma mensagem através de uma caixa de texto e de um botão «Enviar»; a segunda, iniciada ao pressionar o botão, apresenta essa mensagem ao utilizador.

2.1.1. Criar um projecto Android

O projecto foi criado a partir do ecrã inicial do Android Studio (Figura 1). Nas versões actuais, o modelo «Empty Activity» indicado no enunciado gera um projecto em Kotlin com Jetpack Compose, sem layouts XML; para manter a correspondência com o enunciado (layouts em XML e código em Java), escolheu-se o modelo «Empty Views Activity» (Figura 2). Esta mudança ficou registada no histórico do repositório: o projecto partiu do modelo Compose e foi depois refactorizado para Empty Views Activity e, em seguida, para Java.

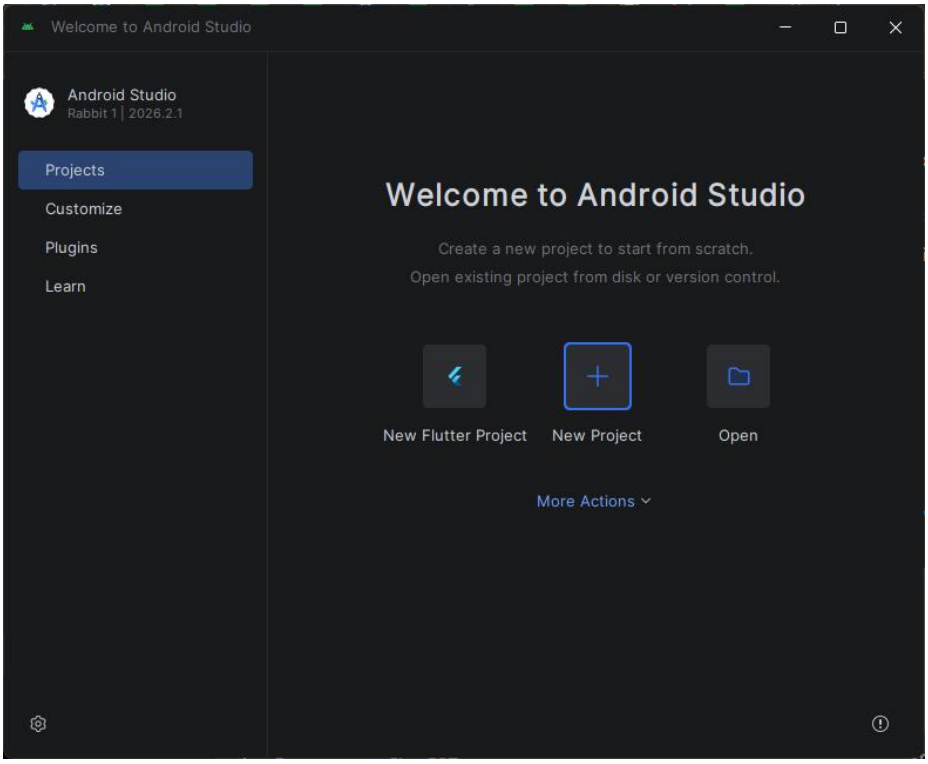


Figura 1 **–** Ecrã de boas-vindas do Android Studio (Rabbit 1 | 2026.2.1), ponto de partida para a criação de um novo projecto

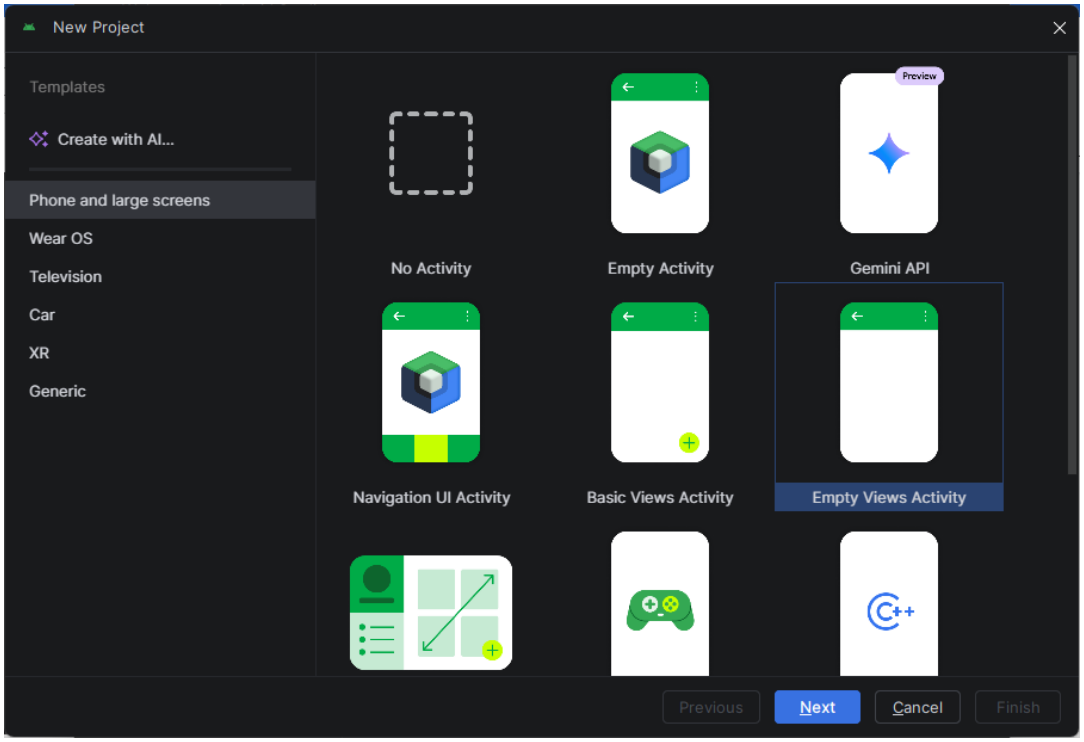


Figura 2 **–** Selecção do modelo Empty Views Activity, que gera uma actividade com layout definido em XML

Na configuração (Figura 3) definiram-se o nome *Hello World*, o pacote `ao.co.isptec.aplm.exercise_1`, a linguagem Java e o SDK mínimo API 23 (Android 6.0); os scripts de construção ficaram em Kotlin DSL (`build.gradle.kts`), a opção recomendada pelo IDE. A Figura 4 mostra a árvore de directórios gerada e a classe `MainActivity` inicial.

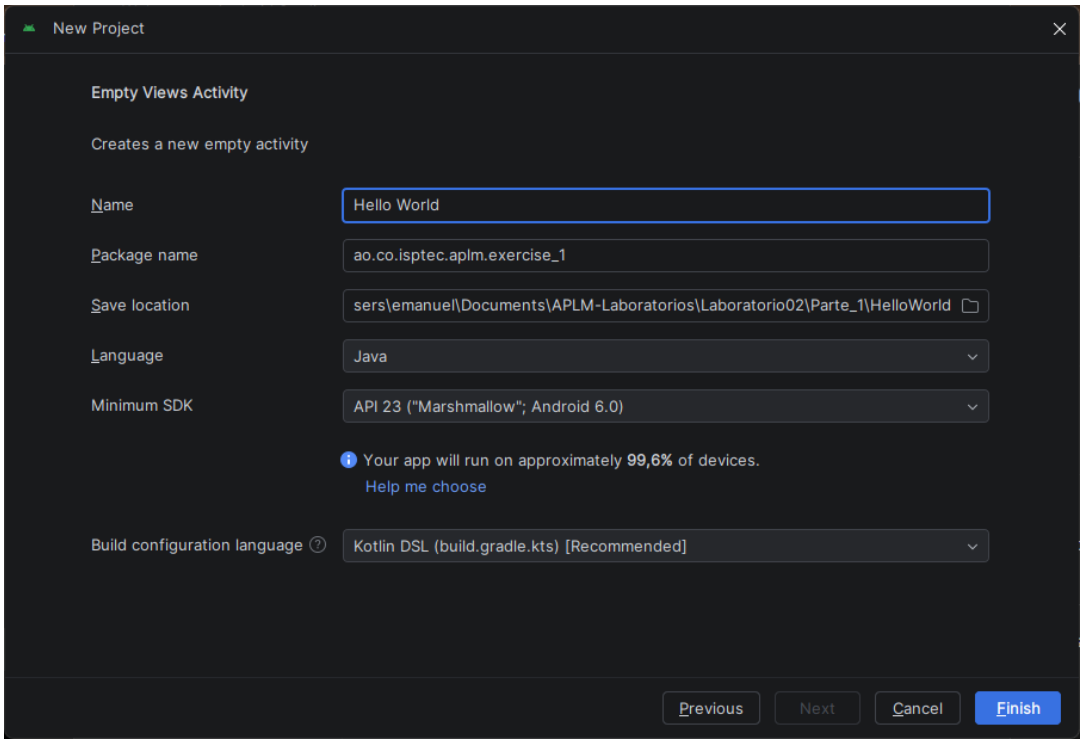


Figura 3 **–** Configuração do projecto Hello World: pacote ao.co.isptec.aplm.exercise_1, linguagem Java e SDK mínimo API 23

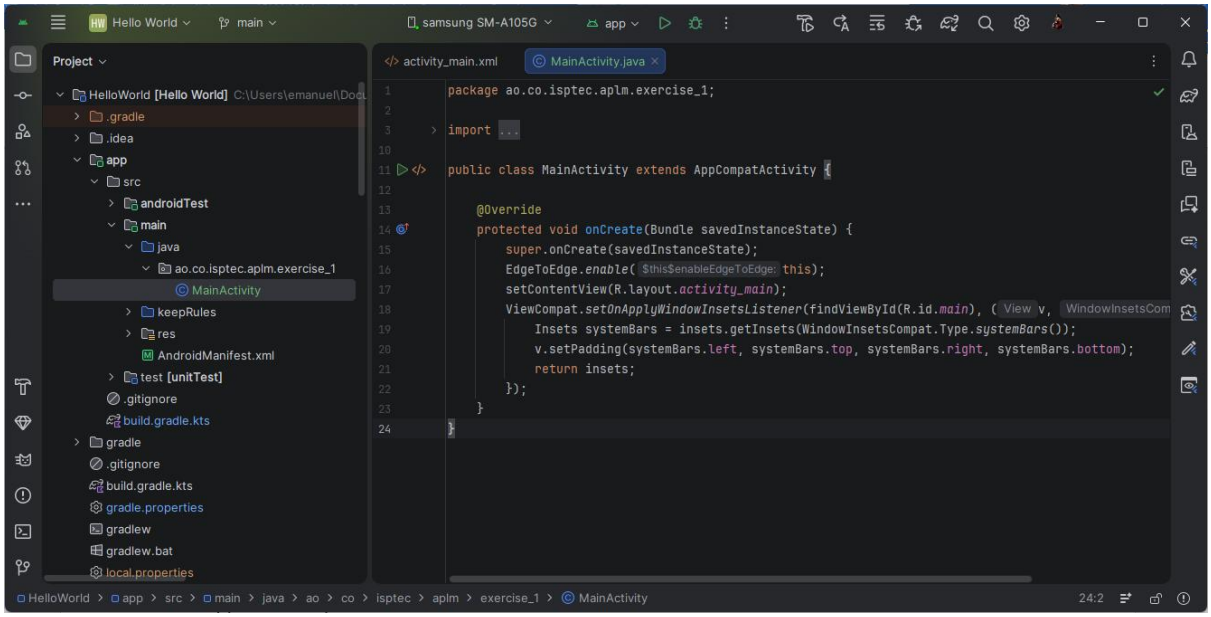


Figura 4 **–** Árvore de directórios do projecto na vista Project e classe MainActivity gerada pelo modelo

1) Qual é a função dos ficheiros `activity_main.xml`, `MainActivity.java`, `AndroidManifest.xml` e `build.gradle` do módulo app?

R:

`activity_main.xml` (app/src/main/res/layout) - recurso que descreve, em XML, o layout da actividade principal, isto é, a árvore de Views apresentada no ecrã.

`MainActivity.java` (app/src/main/java/ao/co/isptec/aplm/exercise_1) - código da actividade principal; no `onCreate()` associa o layout através de `setContentView(R.layout.activity_main)` e trata a interacção do utilizador.

`AndroidManifest.xml` (app/src/main) - esqueleto da aplicação: declara o ícone, o tema e todos os componentes, indicando com o `intent-filter` MAIN/LAUNCHER qual é a actividade de arranque.

`build.gradle.kts` (app) - script Gradle do módulo: aplica o plugin Android e define `namespace`/`applicationId`, `minSdk`, `targetSdk`, versão e dependências (AppCompat, ConstraintLayout, Material).

2.1.2. Executar a aplicação

A aplicação foi executada num dispositivo físico - um Samsung SM-A105G conectado a mesma rede que o computador utilizado em desenvolvimento, com a depuração via WI-FI activa nas Opções do programador. Ao seleccionar o dispositivo e executar **Run**, o Android Studio compila o APK, instala-o via ADB e inicia a `MainActivity`; o espelhamento do ecrã no painel Running Devices mostra o TextView «Hello World!» criado pelo modelo (Figura 5).

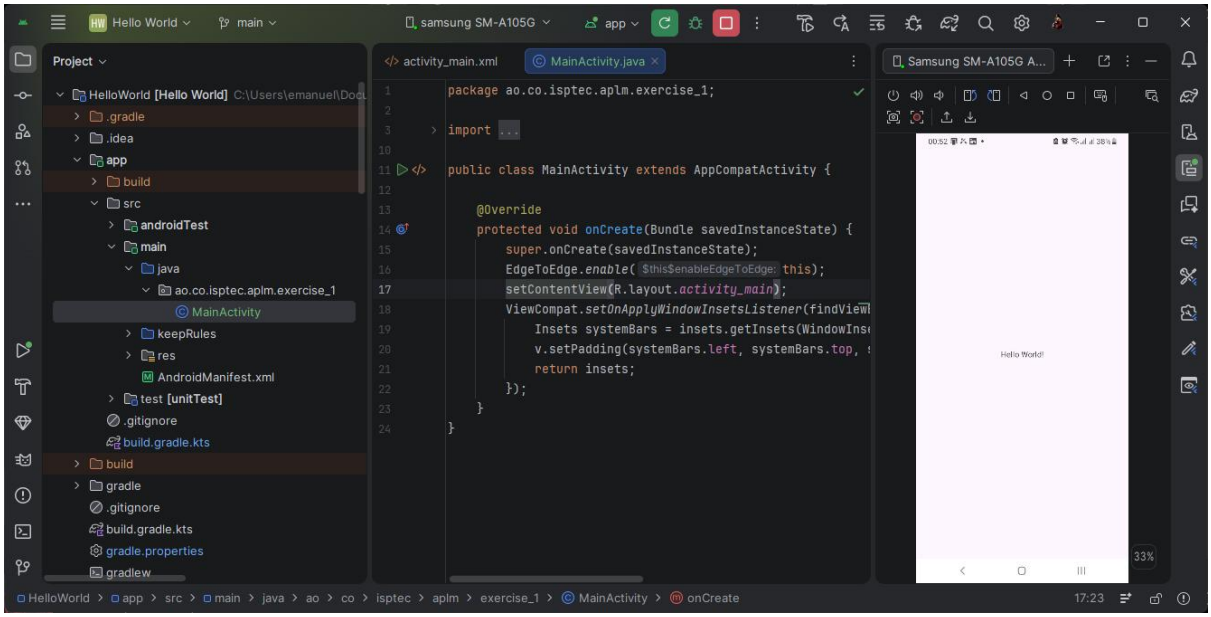


Figura 5 **–** Primeira execução da aplicação no Samsung SM-A105G, espelhado no painel Running Devices do Android Studio

2.1.3. Debug da aplicação

No final do método `onCreate()` da `MainActivity` acrescentou-se a seguinte instrução de registo (com o respectivo `import android.util.Log;`):

```
Log.d("MainActivity", "Minha primeira mensagem de depuração.");
```

Depois de reimplantar a aplicação, a mensagem surge no Logcat com o nível D (*debug*) e a tag `MainActivity`, filtrando pelo pacote `ao.co.isptec.aplm.exercise_1`. Colocou-se ainda um ponto de interrupção na linha do `Log.d` e executou-se a aplicação em modo Debug: a execução suspende nessa linha e o depurador mostra os valores das variáveis em linha (`savedInstanceState: null`, por se tratar do primeiro arranque da actividade), sendo retomada a partir dos controlos do IDE (Figura 6).

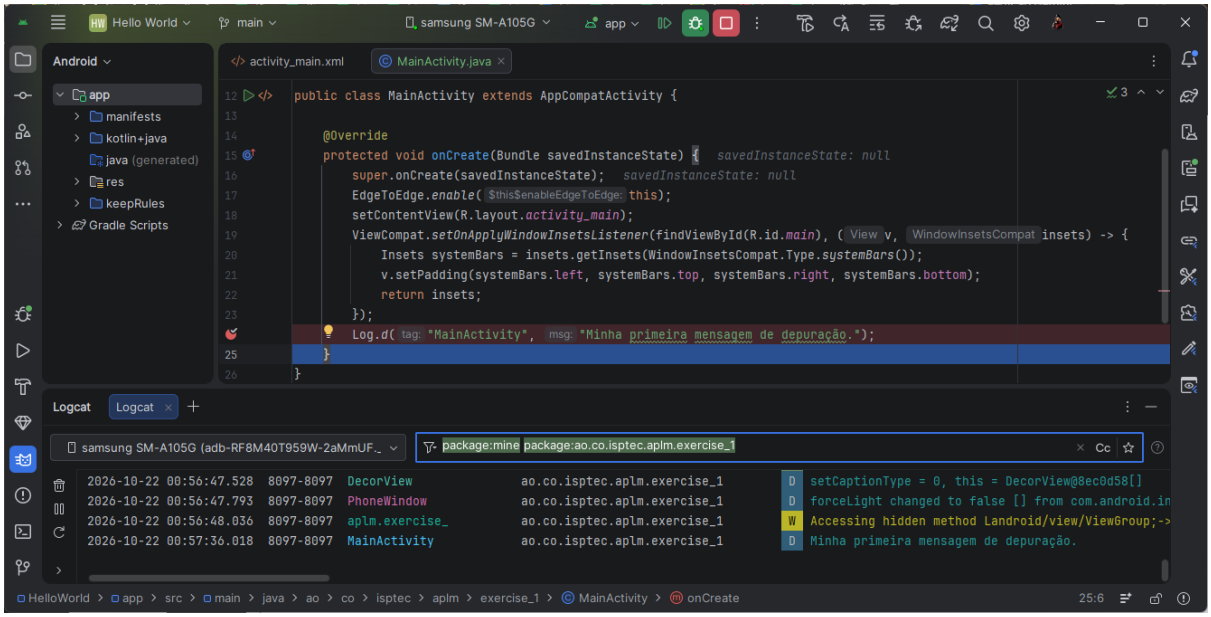


Figura 6 **–** Ponto de interrupção na linha do Log.d e mensagem «Minha primeira mensagem de depuração.» no Logcat, filtrada pelo pacote da aplicação

2.1.4. Construção de uma interface do utilizador simples

A actividade principal deve conter uma caixa de texto e um botão dispostos lado a lado. Na árvore da UI (Figura 7a), a raiz é o `ConstraintLayout` gerado pelo modelo (`@+id/main`, usado para aplicar os *insets* do modo edge-to-edge); dentro dele, um `LinearLayout` horizontal agrupa um `EditText` e um `Button`. No `EditText`, `layout_width="0dp"` com `layout_weight="1"` faz com que a caixa ocupe todo o espaço horizontal deixado livre pelo botão (`wrap_content`). Os textos foram externalizados em `res/values/strings.xml` (`message_hint` e `send`), em vez de literais no layout (Figura 8).

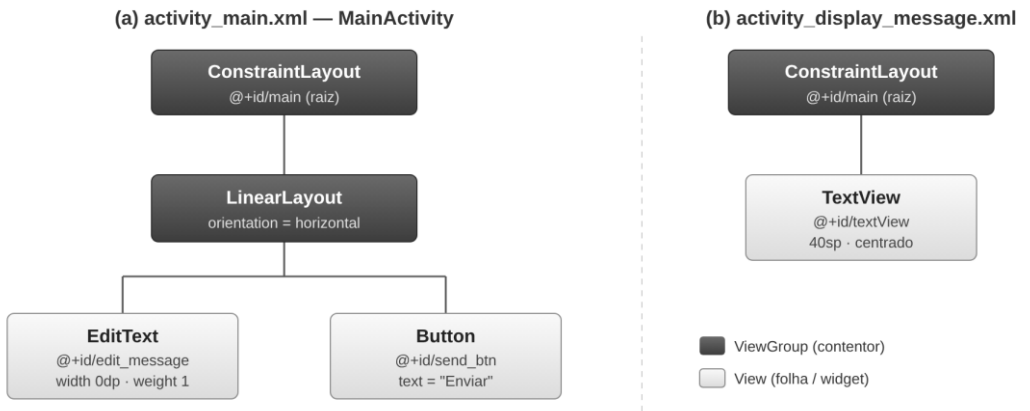


Figura 7 **–** Árvore da UI das actividades da aplicação Hello World: (a) MainActivity; (b) DisplayMessageActivity

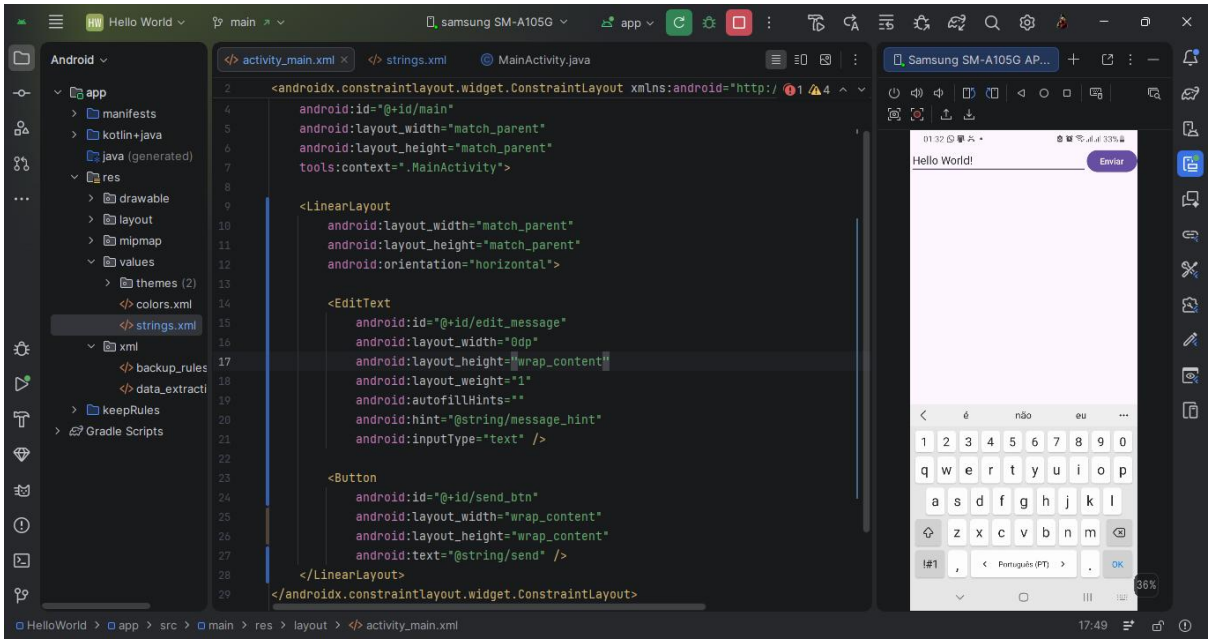


Figura 8 **–** Layout activity_main.xml com o LinearLayout horizontal (EditText + Button) e o respectivo resultado no dispositivo

2.1.5. Iniciar outra actividade

A segunda actividade foi criada com File > New > Activity > Empty Views Activity, com o nome `DisplayMessageActivity`; o assistente gera a classe Java, o layout `activity_display_message.xml` e a declaração no `AndroidManifest.xml`. O código fornecido no enunciado foi adaptado à versão actual da plataforma: as classes `android.support.v7` deram lugar às equivalentes AndroidX (`androidx.appcompat`), o listener anónimo foi substituído por uma expressão lambda e manteve-se o suporte edge-to-edge do modelo. O tratamento do clique na `MainActivity` ficou assim:

```
sendButton.setOnClickListener(view -> {
    String message = editText.getText().toString();
    Intent intent = new Intent(MainActivity.this, DisplayMessageActivity.class);
    intent.putExtra(EXTRA_MESSAGE, message);
    startActivity(intent);
});
```

A `DisplayMessageActivity` lê a mensagem recebida e apresenta-a num `TextView` de 40sp centrado no ecrã (Figura 9). A Figura 10 mostra o fluxo completo no dispositivo: o toque em «Enviar» na primeira actividade dá origem à segunda.

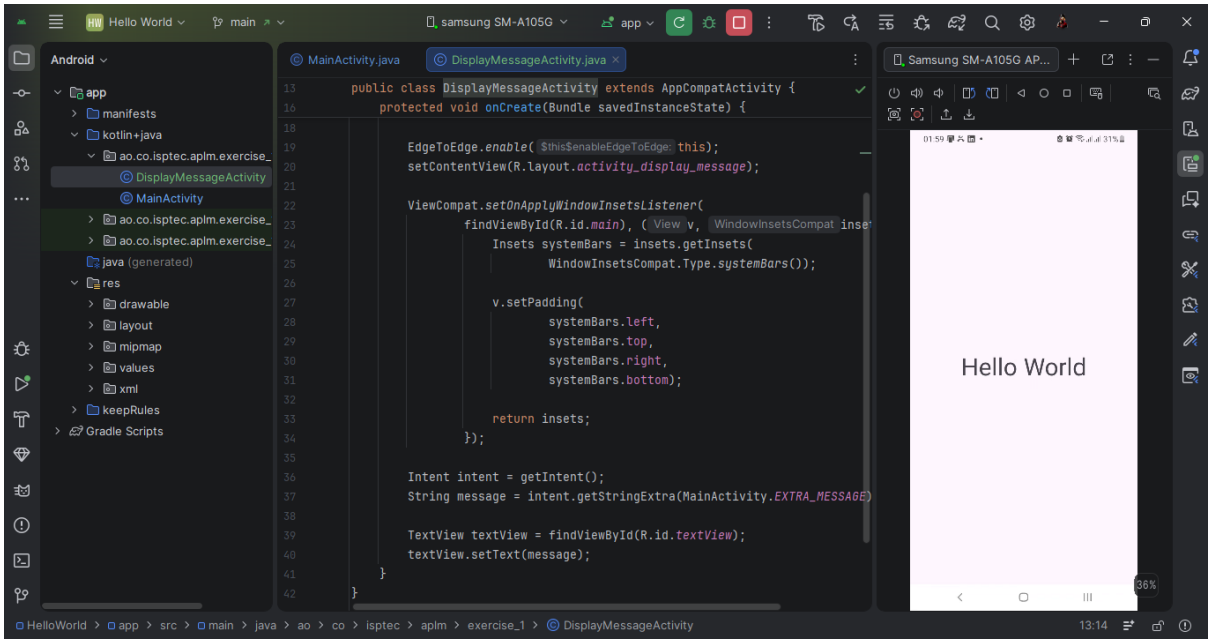
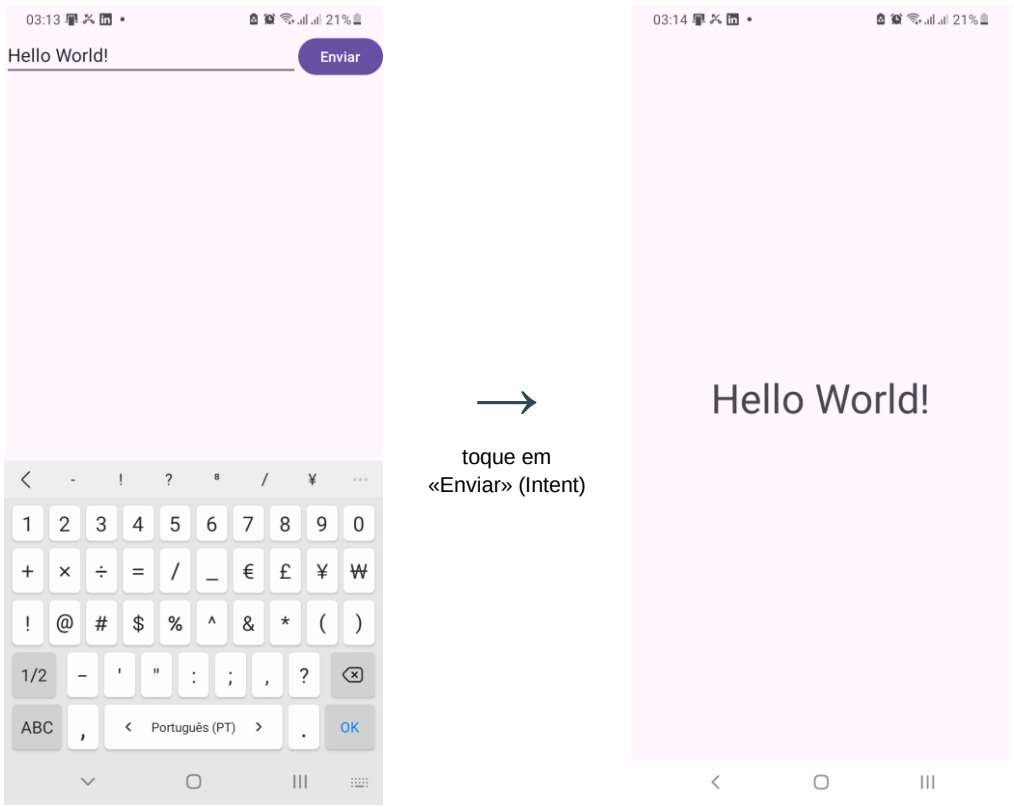


Figura 9 **–** Implementação da DisplayMessageActivity: leitura do extra EXTRA_MESSAGE e apresentação da mensagem no TextView



(a) `MainActivity` **(b)** `DisplayMessageActivity`

Figura 10 **–** Navegação entre actividades: (a) mensagem «Hello World!» escrita na MainActivity; (b) DisplayMessageActivity iniciada pelo toque em «Enviar», apresentando a mensagem recebida

2.1.6. Questões do Exercício I

1) O que alterou no ficheiro `AndroidManifest.xml`?

R:

Foi acrescentada a declaração da nova actividade: `<activity android:name=".DisplayMessageActivity" android:exported="false" />`, gerada pelo assistente. Sem ela, o sistema não consegue iniciar a actividade (`ActivityNotFoundException`). Com `exported="false"`, só a própria aplicação a pode abrir; a `MainActivity` mantém o `intent-filter` MAIN/LAUNCHER, continuando a ser o ponto de entrada.

2) Como o evento clique é tratado?

R:

Regista-se um listener no botão com `setOnClickListener()`, aqui como lambda que implementa `View.OnClickListener`. Ao clicar, o sistema invoca `onClick()`, que lê o texto do `EditText`, cria um Intent explícito para a `DisplayMessageActivity`, anexa a mensagem com `putExtra()` e chama `startActivity()`. Em alternativa, poder-se-ia usar o atributo `android:onClick` no XML, com um método público na actividade.

3) Como identificar e obter ou definir componentes de interface do código Java?

R:

Cada componente recebe no XML um identificador, e.g. `android:id="@+id/edit_message"`, a partir do qual o build gera a constante `R.id.edit_message`. No código obtém-se a referência com `findViewById(R.id.edit_message)` e lê-se ou define-se o seu estado com os respectivos métodos, e.g. `editText.getText().toString()` ou `textView.setText(message)`.

4) Qual é a finalidade da `intent` da variável criada na classe `DisplayMessageActivity`?

R:

`getIntent()` devolve o Intent que iniciou a actividade. Serve para recuperar os dados enviados pela `MainActivity` - `getStringExtra(MainActivity.EXTRA_MESSAGE)` e apresentá-los no `TextView`. A chave `EXTRA_MESSAGE`, prefixada com o nome do pacote, evita colisões com extras de outras aplicações.

2.2. Exercício II – Desmistificando Gradle

O Gradle é o sistema de automação de construção usado pelo Android. Neste exercício utilizou-se primeiro a distribuição autónoma gradle-5.2.1, instalada localmente com a variável `GRADLE_HOME` a apontar para a pasta de instalação e o respectivo directório `bin` acrescentado ao `PATH`, e depois a distribuição que acompanha o projecto Android (Gradle Wrapper).

2.2.1. Comandos básicos do Gradle

Com a instalação concluída, executaram-se os comandos de exploração indicados no enunciado (Figuras 11 a 13). O comando `gradle -v` confirma a versão 5.2.1 a correr sobre a JVM 11 em Windows 11.

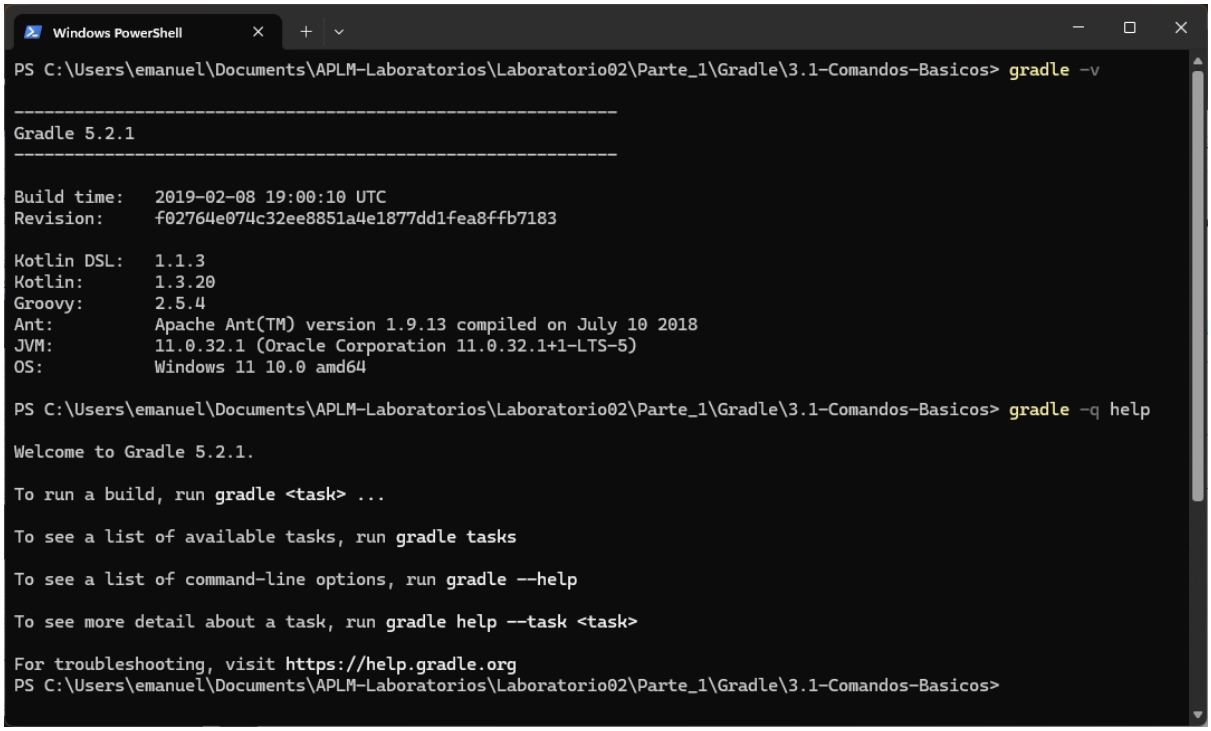


Figura 11 **–** Execução de gradle -v e gradle -q help: Gradle 5.2.1 sobre a JVM 11

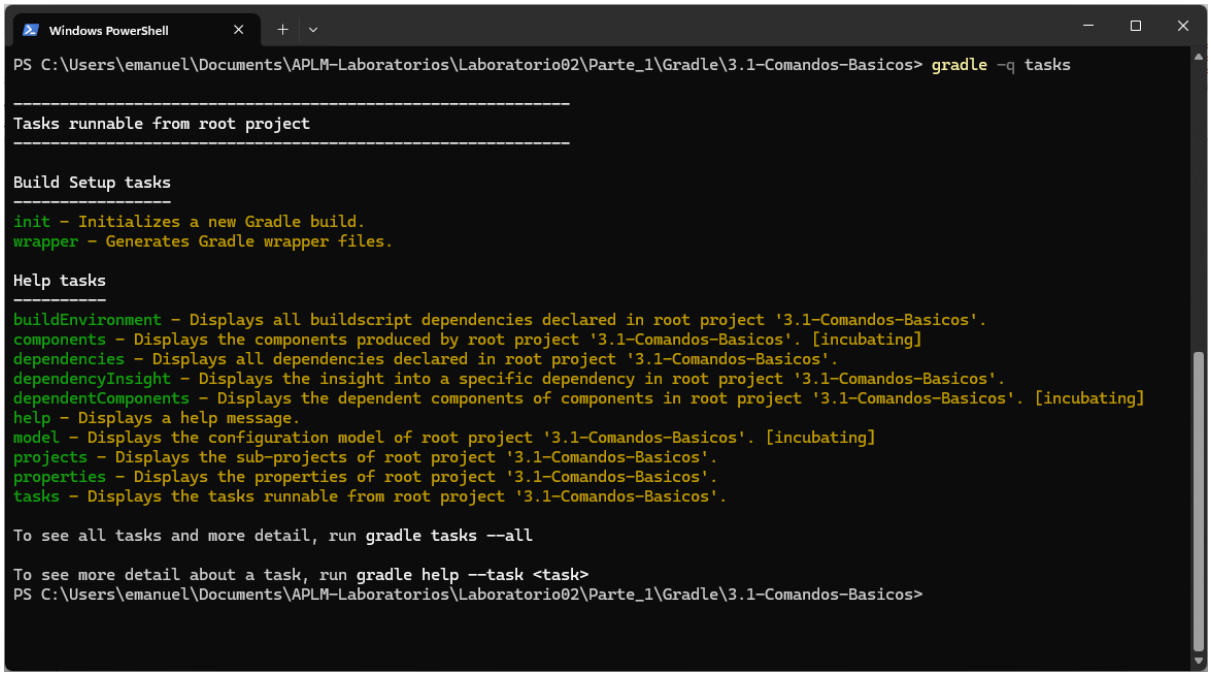


Figura 12 **–** Saída de gradle -q tasks numa pasta sem build.gradle: apenas tarefas de Build Setup e Help

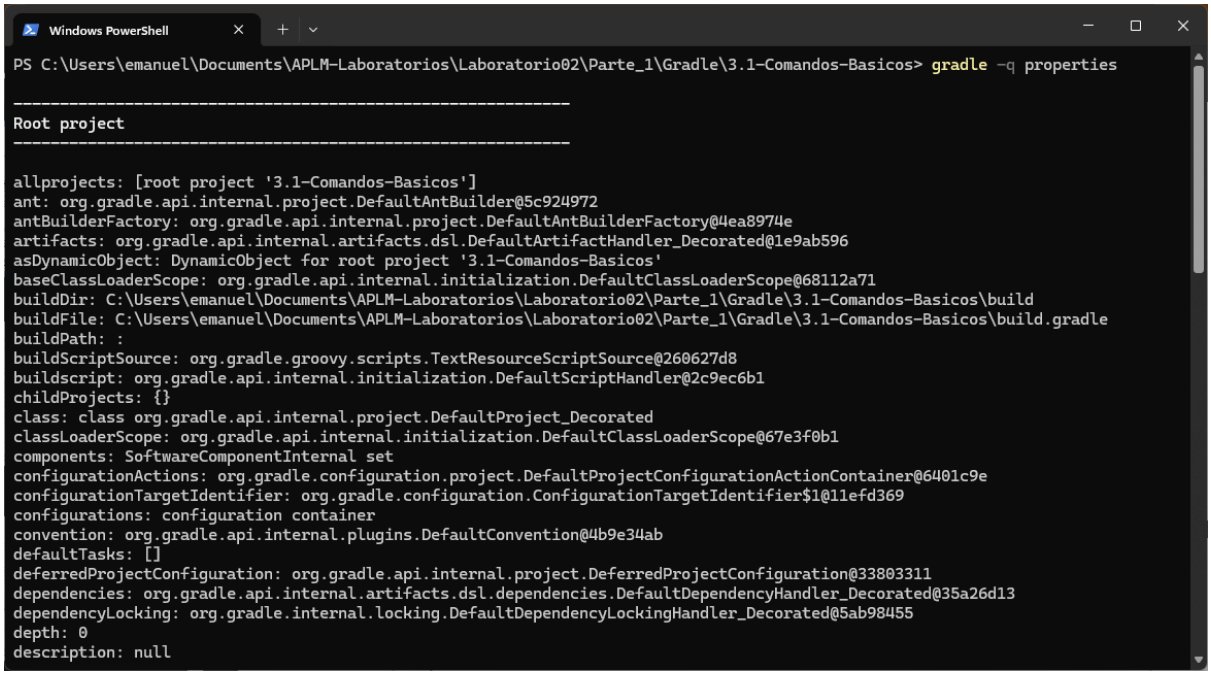


Figura 13 **–** Excerto da saída de gradle -q properties com as propriedades do projecto raiz

Na pasta `exemplo1` criou-se o ficheiro `build.gradle` com a tarefa `compileTask`; com `gradle -q tasks --all`, a tarefa surge no grupo *Other tasks* e a sua execução imprime «compiling...» (Figura 14). O ficheiro foi depois modificado para a versão final:

```
defaultTasks 'buildTask'

task compileTask {
    doLast {
        System.out.println "compiling..."
    }
}

task buildTask (dependsOn:compileTask) {
    doLast {
        System.out.println "building..."
    }
}
```

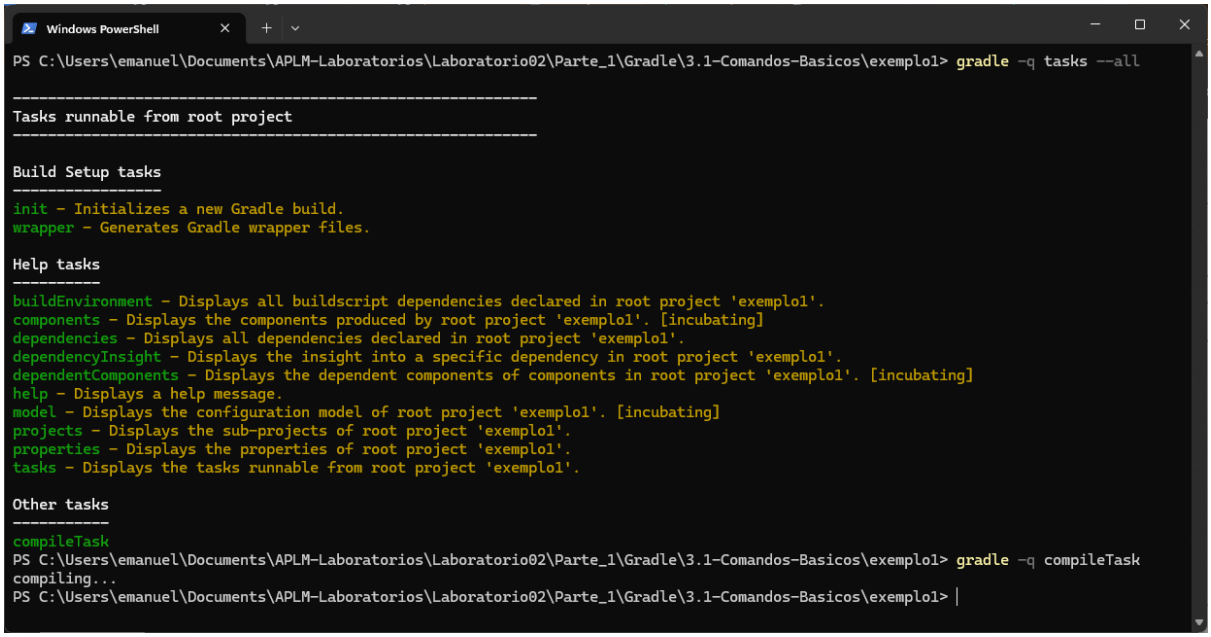


Figura 14 **–** Tarefa compileTask listada em Other tasks (gradle -q tasks --all) e respectiva execução

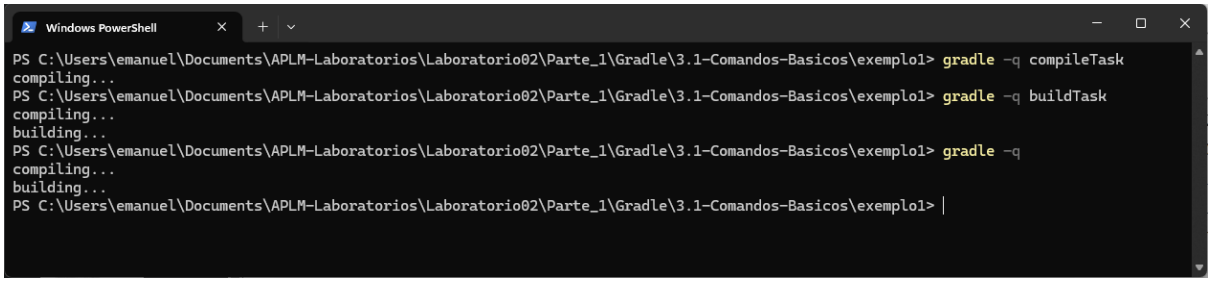


Figura 15 **–** Execução de compileTask, buildTask e da tarefa por omissão (gradle -q)

1) O que fazem os comandos `gradle -v`, `gradle -q help`, `gradle -q tasks` e `gradle -q properties`?

R:

`gradle -v` mostra a versão do Gradle e do ambiente em que corre (Groovy, Kotlin, Ant, JVM e sistema operativo). A opção `-q` (*quiet*) suprime as mensagens de registo, deixando apenas os erros e a saída das tarefas. `help` apresenta a ajuda básica de utilização; `tasks` lista as tarefas executáveis no projecto (sem `build.gradle`, apenas as de Build Setup e Help); `properties` lista as propriedades do projecto, como `buildDir`, `buildFile` ou `defaultTasks`.

2) Como se interpreta a saída de `gradle -q compileTask`, `gradle -q buildTask` e `gradle -q`?

R:

`compileTask` executa apenas a sua acção `doLast` e imprime «compiling...». `buildTask` declara `dependsOn: compileTask`, pelo que o Gradle constrói o grafo de tarefas e executa primeiro a dependência: «compiling...» seguido de «building...». Sem tarefa indicada, `gradle -q` executa as `defaultTasks`, aqui `buildTask` - e o resultado é idêntico ao anterior.

2.2.2. Simples projecto Java com Gradle Java plugin

Criou-se o projecto `exemplo2` com a classe de domínio `Quote` (identificador, autor e texto de uma citação) em `src/main/java/ao/co/isptec/aplm/quota`. O `build.gradle` final, já com o plugin *application*, ficou assim:

```
apply plugin: 'java'
apply plugin: 'application'

archivesBaseName = "quote"
version = '1.0-FINAL'

repositories {
    mavenCentral()
}

dependencies {
    compile group: 'org.apache.commons', name: 'commons-lang3', version: '3.3.2'
    testCompile group: 'junit', name: 'junit', version: '4.+'
}

mainClassName = 'ao.co.isptec.aplm.quote.Quote'
```

A Figura 16 mostra as tarefas acrescentadas pelo plugin Java e a Figura 17 o resultado de `gradle build`. Em seguida implementou-se o método `main` pedido, adaptado aos métodos da classe fornecida (`setWho`/`setWhat` em vez de `setAutor`/`setTexto`); a sua execução com `gradle run` é apresentada na Figura 18.

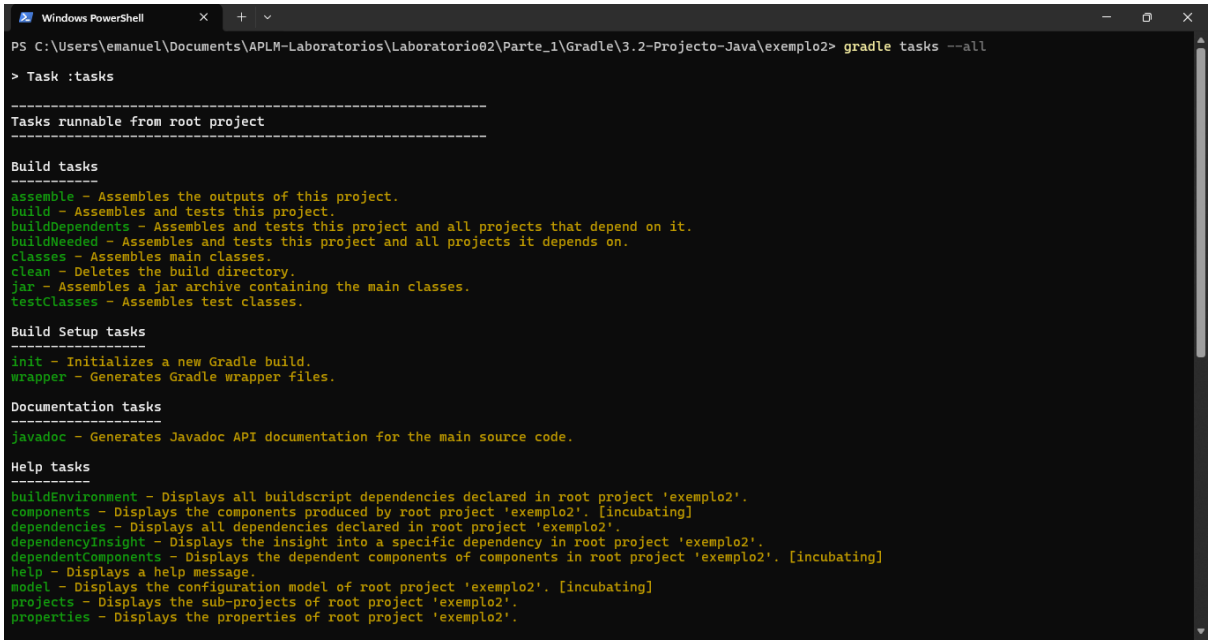


Figura 16 **–** Tarefas disponibilizadas pelo plugin Java (gradle tasks --all) no projecto exemplo2

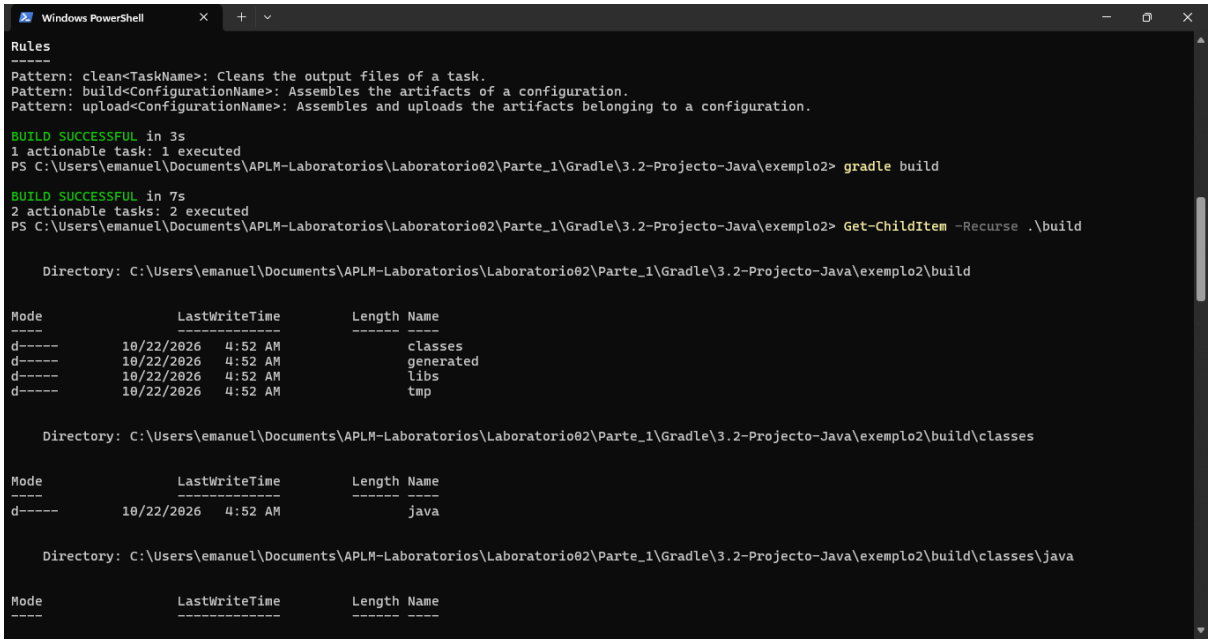


Figura 17 **–** Execução de gradle build e estrutura do directório build gerado (classes, generated, libs e tmp)

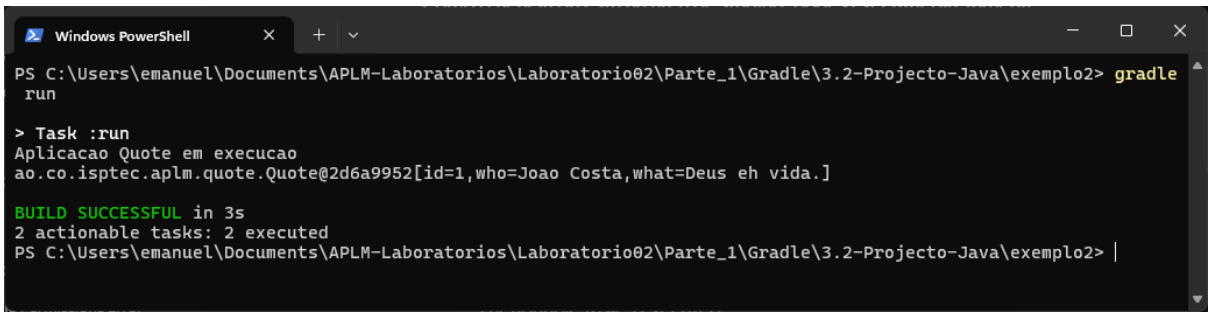


Figura 18 **–** Execução de gradle run: a classe Quote é executada e o objecto é impresso através do ToStringBuilder

1) Qual é o propósito da declaração `apply plugin: "java"` em `build.gradle`?

R:

Aplica o plugin Java, que acrescenta ao projecto as convenções de estrutura (`src/main/java`, `src/test/java`), as configurações de dependências (`compile`, `testCompile`, …) e as tarefas necessárias para compilar, testar e empacotar código Java (`compileJava`, `test`, `jar`, `build`, …), sem ser preciso escrevê-las manualmente.

2) Que tarefas define o plugin Java e qual é o resultado de `gradle build`?

R:

Para além das tarefas de Help e Build Setup, surgem os grupos Build (`assemble`, `build`, `classes`, `clean`, `jar`, `testClasses`, …), Documentation (`javadoc`) e Verification (`check`, `test`), bem como regras como `clean<TaskName>`. `gradle build` é uma tarefa agregadora: compila (`compileJava` → `classes`), empacota (`jar` → `assemble`) e corre os testes (`test` → `check`). O resultado é a pasta `build/`, com os `.class` em `build/classes/java/main` e o JAR em `build/libs`, cujo nome resulta de `archivesBaseName` e `version` (`quote-1.0-FINAL.jar`).

3) Por que o `Quote.java` é colocado nesse directório específico?

R:

Porque `src/main/java` é, por convenção do plugin Java (herdada do Maven), o directório do código-fonte de produção: respeitando-a, não é necessário configurar *source sets*. Dentro dele, os subdirectórios espelham o pacote da classe. No repositório a pasta chama-se `quota`, mas o pacote declarado é `ao.co.isptec.aplm.quote`; compila porque o `javac` não impõe essa correspondência, embora a convenção recomende alinhá-los.

4) Qual é o significado dos blocos `dependencies` e `repositories` no ficheiro `build.gradle`?

R:

`repositories` indica onde o Gradle procura e descarrega as bibliotecas externas — aqui, o Maven Central. `dependencies` declara essas bibliotecas e o âmbito em que são usadas: `compile` para a commons-lang3 3.3.2, necessária para compilar e executar (fornece o `ToStringBuilder` usado no `toString()`), e `testCompile` para o JUnit 4, disponível apenas para os testes. Nas versões actuais do Gradle, estas configurações foram substituídas por `implementation` e `testImplementation`.

5) O que acontece ao executar `gradle run`?

R:

O plugin *application* acrescenta a tarefa `run` (bem como `startScripts`, `distZip` e `installDist`), que depende de `classes`: o Gradle compila o projecto e lança a JVM com a classe definida em `mainClassName` e o classpath de execução, que inclui a commons-lang3. O `main` imprime «Aplicacao Quote em execucao» e o objecto formatado por reflexão: `Quote@2d6a9952[id=1,who=Joao Costa,what=Deus eh vida.]`.

2.2.3. Múltiplos projectos Java

Descompactou-se o projecto `example3` (ficheiro `exercise-ii.zip`) e analisaram-se os seus scripts. O `settings.gradle` declara os subprojectos e o `build.gradle` raiz aplica, através do bloco `subprojects`, a configuração comum a todos:

```
// settings.gradle
include ":api", ":common", ":app"

// build.gradle (raiz)
subprojects {
    apply plugin: "java"
    repositories { mavenCentral() }
    dependencies { testCompile "junit:junit:4+" }
}
```

As Figuras 19 e 20 mostram a listagem dos subprojectos, a construção global e a construção individual de cada módulo.

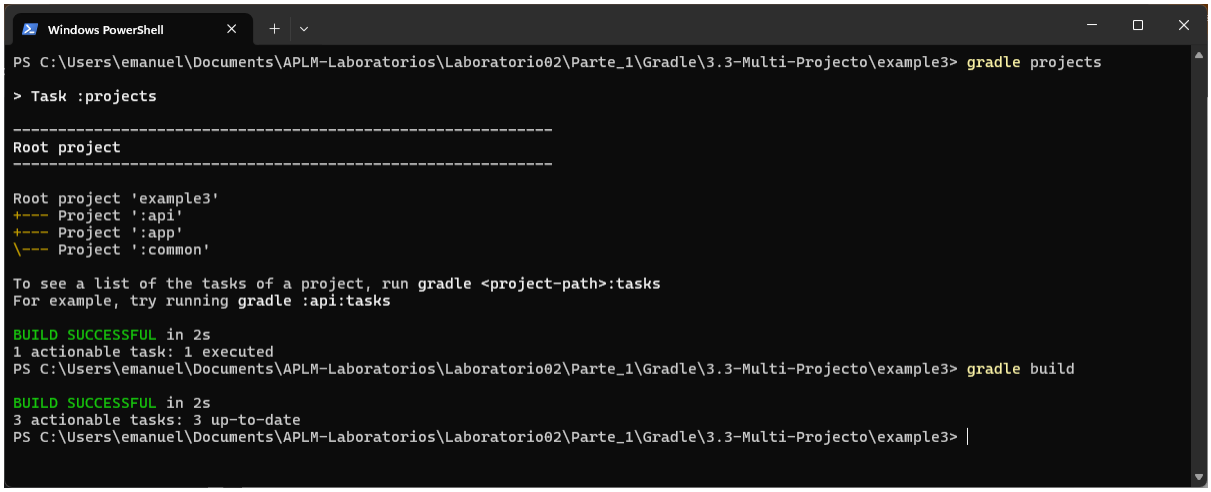


Figura 19 **–** Listagem dos subprojectos (gradle projects) e construção de todo o projecto (gradle build)

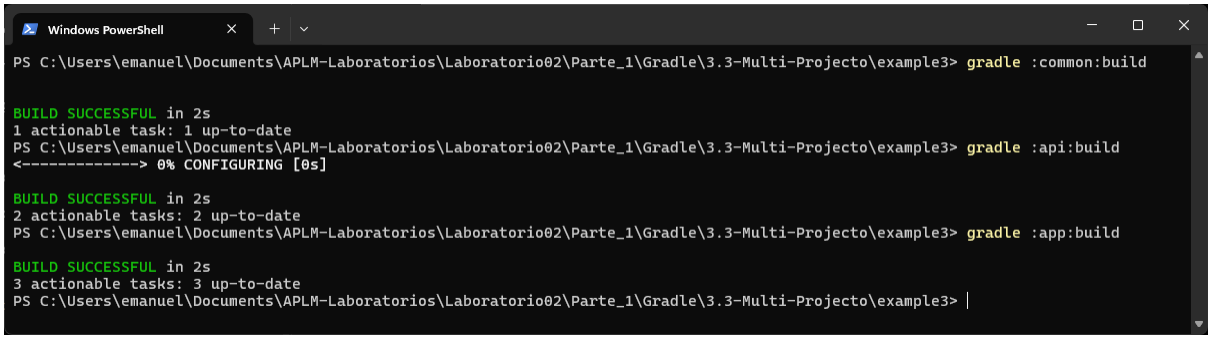


Figura 20 **–** Construção individual de cada subprojecto: :common, :api e :app

1) Em quais subprojectos este projecto está dividido e como se executa o Gradle para construir (i) todo o projecto e (ii) cada subprojecto individualmente?

R:

Está dividido em três subprojectos — `api`, `app` e `common` — declarados no `settings.gradle`. (i) Para construir tudo, executa-se `gradle build` na raiz, que corre a tarefa em todos os subprojectos. (ii) Para um subprojecto usa-se o caminho qualificado da tarefa: `gradle :common:build`, `gradle :api:build` e `gradle :app:build` (ou `gradle build` dentro da pasta do subprojecto). Como o Gradle resolve as dependências entre projectos, o número de tarefas executadas cresce de 1 para 3, reflectindo a construção prévia dos projectos de que cada um depende.

2) Quais são as dependências de cada subprojecto e qual é o grafo de dependências do projecto?

R:

`common` depende apenas da commons-lang3 3.3.2. `api` depende do subprojecto `common`, da commons-lang3 e do log4j 1.2.17. `app` depende dos subprojectos `common` e `api` e do log4j. Todos herdam o JUnit 4 (`testCompile`) do projecto raiz. O grafo (Figura 21) é acíclico, com `common` na base e `app` no topo, pelo que a ordem de construção é common → api → app.

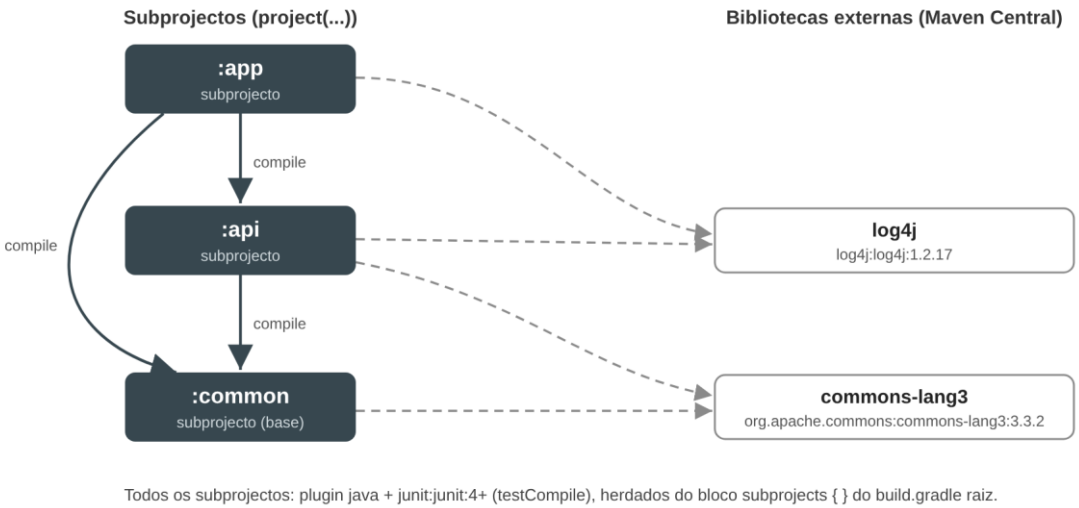


Figura 21 **–** Grafo de dependências do projecto example3: subprojectos (setas contínuas) e bibliotecas externas (setas tracejadas)

2.2.4. Gradle e Android Studio

Voltou-se ao projecto Hello World para observar a integração do Gradle no Android Studio, feita através do Android Gradle Plugin. No painel Gradle do IDE (Figura 22), o projecto surge como uma árvore com o projecto raiz e o módulo `app`; por omissão, a lista completa de tarefas não é gerada durante a sincronização («Task list not built…»), sendo necessário activar essa opção nas definições ou recorrer à linha de comandos.

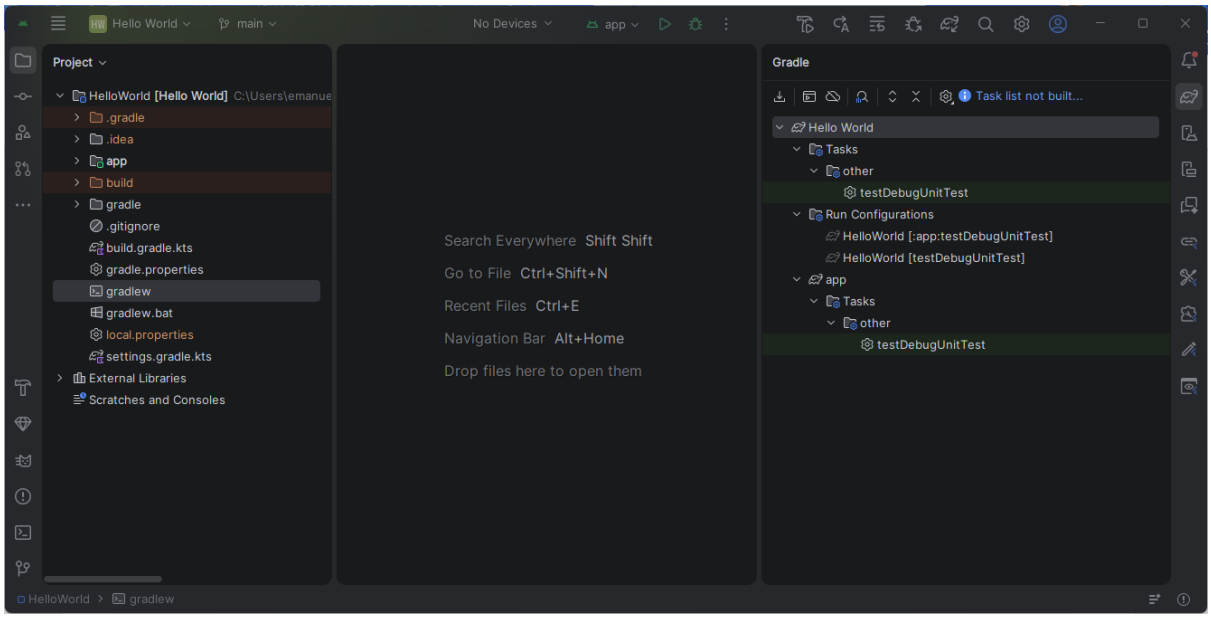


Figura 22 **–** Painel Gradle do Android Studio com a estrutura do projecto Hello World e do módulo app

As tarefas foram executadas num terminal externo através do Gradle Wrapper (`gradlew.bat`), que descarrega e utiliza a versão do Gradle fixada pelo projecto - 9.6.0, independentemente da distribuição 5.2.1 instalada (Figuras 23 e 24). A construção completa executou 86 tarefas, incluindo a análise do lint, cujo relatório foi gerado em `app/build/reports`.

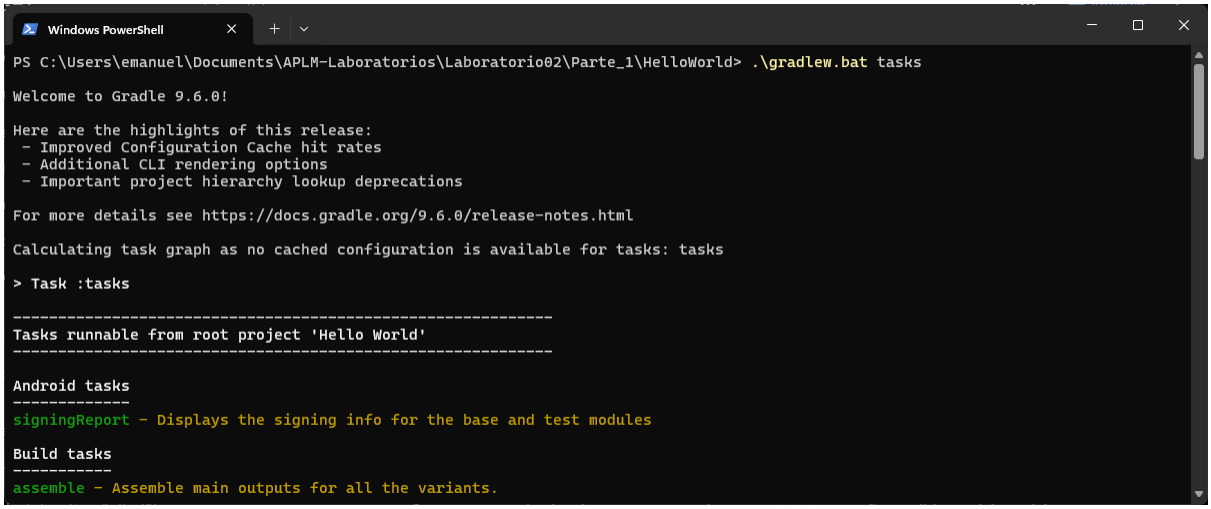


Figura 23 **–** Listagem de tarefas do projecto Android com o Gradle Wrapper (gradlew.bat tasks), que utiliza o Gradle 9.6.0

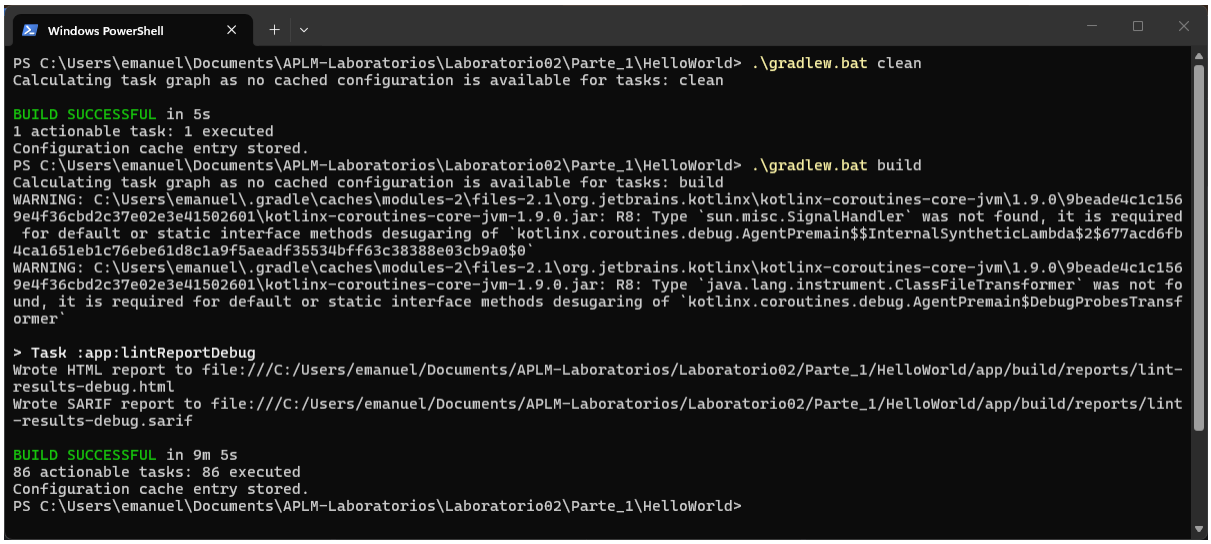


Figura 24 **–** Limpeza (clean) e construção completa (build) do projecto Hello World num terminal externo, com o Gradle Wrapper

1) Que ficheiros da estrutura interna do projecto são relevantes para o Gradle e qual é a sua função?

R:

`settings.gradle.kts` - define o nome do projecto raiz, os módulos incluídos (`include(":app")`) e os repositórios de plugins e de dependências.

`build.gradle.kts` (raiz) - configuração comum a todos os módulos; declara o Android Gradle Plugin sem o aplicar (`apply false`).

`app/build.gradle.kts` - configuração do módulo: plugin `com.android.application`, `namespace`, `compileSdk`/`minSdk`/`targetSdk`, tipos de build e dependências.

`gradle/libs.versions.toml` - catálogo de versões que centraliza as versões de plugins e bibliotecas, referidas nos scripts como `libs.*`.

`gradle.properties` - propriedades do Gradle e da JVM (e.g. `org.gradle.jvmargs` e a *configuration cache*).

`gradlew`, `gradlew.bat` e `gradle/wrapper/` - Gradle Wrapper: scripts e configuração (`distributionUrl`) que descarregam e executam a versão exacta do Gradle exigida pelo projecto.

`local.properties` - caminho local do Android SDK; é específico de cada máquina e não deve ser versionado.

3. Desafios

O desafio corresponde ao Exercício III do enunciado: uma aplicação de lista de tarefas (projecto ListadeTarefas, pacote `ao.co.isptec.aplm.exercise_3`) constituída por uma única actividade. O utilizador escreve uma tarefa numa caixa de texto e, ao confirmar, a tarefa é acrescentada no topo de uma lista. O software assenta em três peças: o layout `activity_main.xml`, que define a interface; a `MainActivity`, que trata a entrada do utilizador; e um `ArrayAdapter`, que liga a colecção de tarefas em memória à lista apresentada.

3.1. Desenho da interface móvel

O wireframe da actividade (Figura 25) organiza dois elementos na vertical, dentro de um `LinearLayout` com *padding* de 16dp: no topo, um `EditText` de uma só linha com a sugestão «Escreva uma tarefa»; por baixo, um `ListView` que, com altura 0dp e peso 1, ocupa todo o espaço restante do ecrã. A entrada é confirmada com a tecla Enter ou com o botão OK do teclado virtual (`imeOptions="actionDone"`), dispensando um botão adicional; cada tarefa é apresentada com o layout padrão `simple_list_item_1`.



Figura 25 **–** Wireframe da actividade principal da aplicação Lista de Tarefas

A Figura 26 mostra a implementação do layout e a sua pré-visualização no editor; a Figura 27 confirma o comportamento no dispositivo.

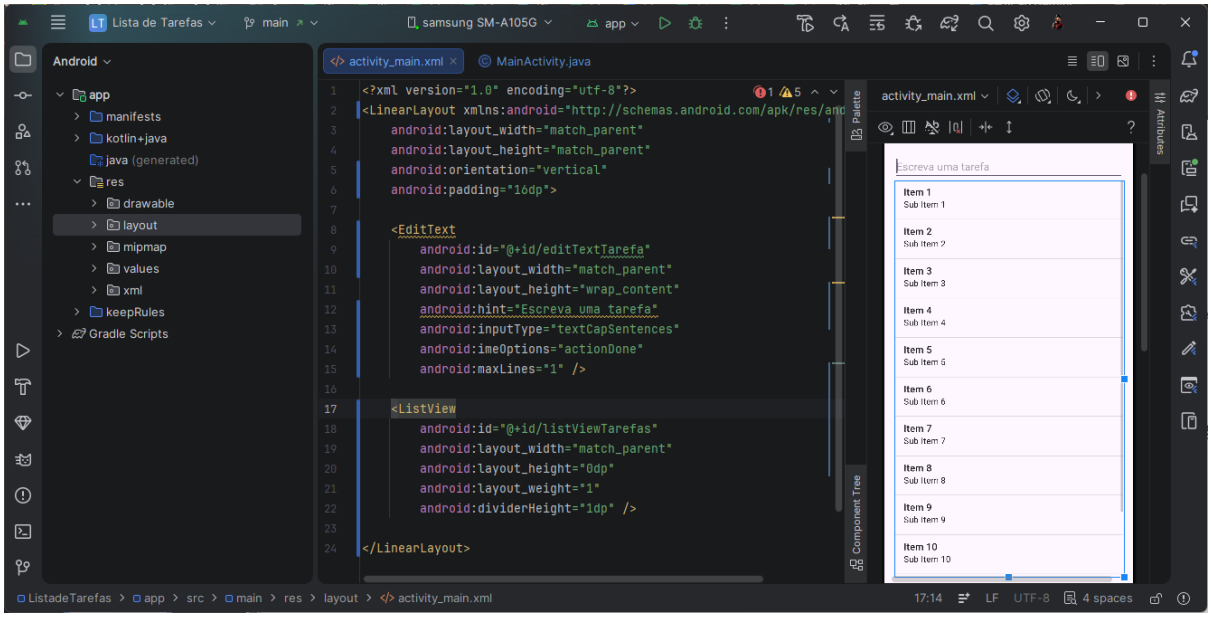
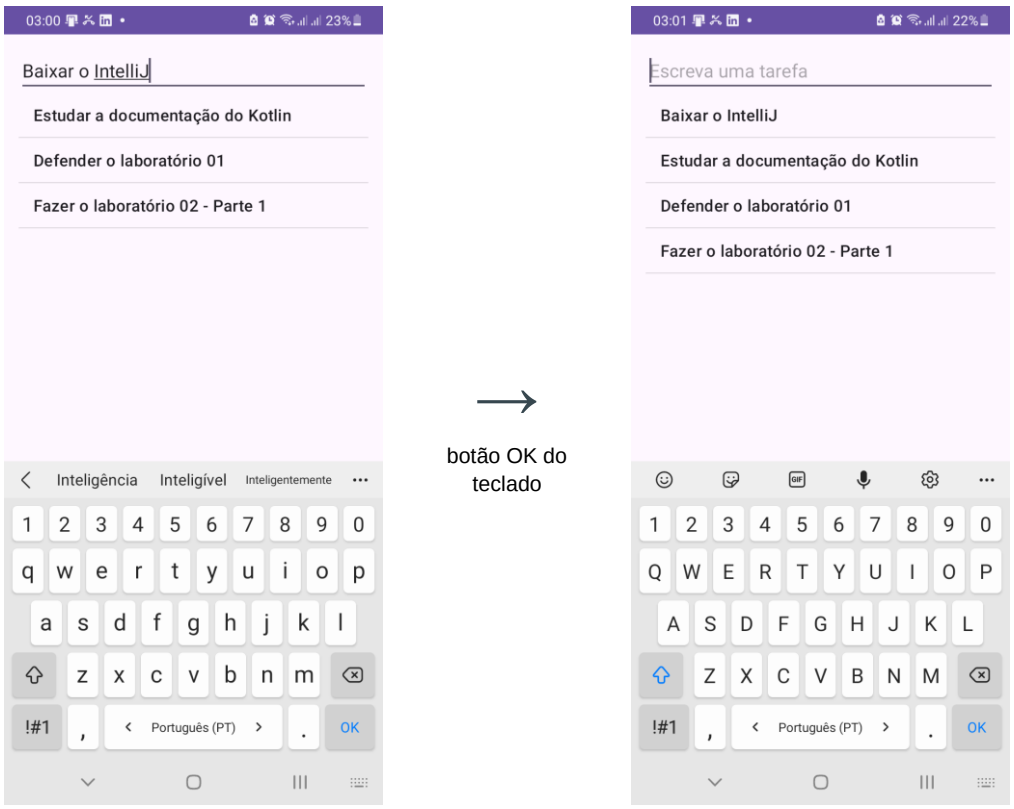


Figura 26 **–** Layout activity_main.xml da Lista de Tarefas (EditText + ListView) e pré-visualização no editor de layouts



(a) antes de confirmar **(b)** após confirmar

Figura 27 **–** Inserção de uma tarefa: (a) texto «Baixar o IntelliJ» escrito no EditText; (b) após confirmar com OK, a tarefa surge no topo da lista e o campo é limpo

3.2. Detalhes de implementação

Arquitectura. A aplicação segue o modelo de actividade única, com separação entre dados e apresentação segundo o padrão *adapter* do Android: o modelo é um `ArrayList<String>` com as tarefas, a vista é o `ListView` e o `ArrayAdapter<String>` faz a ponte entre ambos, criando uma View por elemento. A `MainActivity` actua como controlador: obtém as referências com `findViewById()`, liga o adapter à lista com `setAdapter()` e regista os listeners de entrada.

Estrutura de dados. As tarefas são guardadas num `ArrayList` em memória. Cada nova tarefa é inserida com `add(0, tarefa)`, o que a coloca no índice 0 e, portanto, no topo da lista, sem necessidade de ordenação; o custo O(n) por inserção (deslocamento dos restantes elementos) é irrelevante para listas desta dimensão. Após cada inserção, `notifyDataSetChanged()` avisa o adapter para redesenhar o `ListView`.

Captura da entrada. Seguindo a dica do enunciado, implementou-se `setOnKeyListener()` para a tecla Enter (`KEYCODE_ENTER` em `ACTION_DOWN`, para não duplicar a inserção no `ACTION_UP`). Como os teclados virtuais, em regra, não geram eventos de tecla mas sim acções do IME, registou-se também `setOnEditorActionListener()` para a acção `IME_ACTION_DONE`, é este listener que trata o botão OK no dispositivo real. Ambos delegam no mesmo método:

```
private void adicionarTarefa() {
    String tarefa = editTextTarefa.getText().toString().trim();
    if (tarefa.isEmpty()) {
        return;
    }
    tarefas.add(0, tarefa);           // insere no topo da lista
    adapter.notifyDataSetChanged();   // actualiza o ListView
    editTextTarefa.setText("");       // limpa o campo
}
```

O texto é normalizado com `trim()` e as entradas vazias são ignoradas; no fim, o campo é limpo para a tarefa seguinte. Como limitação, as tarefas existem apenas em memória: perdem-se ao fechar a aplicação ou quando a actividade é recriada (por exemplo, numa rotação do ecrã), o que poderia ser resolvido com `onSaveInstanceState()` ou com persistência local (SharedPreferences ou Room).

4. Referências Bibliográficas

Android. (s.d.). *About Android*. http://android.com/about/

Android Developers. (s.d.). *Android Developers*. http://developer.android.com

Android Developers. (s.d.). *API guides*. http://developer.android.com/guide/components/index.html

Android Developers. (s.d.). *Build your first app*. https://developer.android.com/training/basics/firstapp

Android Developers. (s.d.). *Configure your build*. https://developer.android.com/sdk/installing/studio-build.html

Android Developers. (s.d.). *Create and manage virtual devices*. https://developer.android.com/studio/run/managing-avds

Android Developers. (s.d.). *Layouts in views*. https://developer.android.com/develop/ui/views/layout/declaring-layout

Android Developers. (s.d.). *Package index*. http://developer.android.com/reference/packages.html

Android Developers. (s.d.). *Platform architecture*. https://developer.android.com/guide/platform#art

Android Developers. (s.d.). *Training*. http://developer.android.com/training/index.html

Android Tools Project Site. (s.d.). *Gradle plugin user guide*. http://tools.android.com/tech-docs/new-build-system/user-guide

The Apache Software Foundation. (s.d.). *Apache Ant*. https://ant.apache.org/

Costa, J. (2025). *Laboratório #01: Introdução ao Android* [Enunciado de laboratório, Aplicações Móveis, ano lectivo 2025-26]. Instituto Superior Politécnico de Tecnologias e Ciências (ISPTEC).

Gradle Inc. (s.d.). *The Application plugin*. https://docs.gradle.org/current/userguide/application_plugin.html

Gradle Inc. (s.d.). *Gradle Build Tool*. https://gradle.org/

Gradle Inc. (s.d.). *Gradle documentation*. https://gradle.org/documentation

Gradle Inc. (s.d.). *The Java plugin*. http://gradle.org/docs/current/userguide/java_plugin.html

Gradle Inc. (2019). *Gradle 5.2.1* [Software]. https://services.gradle.org/distributions/gradle-5.2.1-all.zip

Irani, R. (s.d.). *Gradle tutorial: Part 6 — Android Studio + Gradle*. https://rominirani.com/gradle-tutorial-part-6-android-studio-gradle-c828c5639bb

5. Repositório GitHub

Repositório: github.com/SW-Wanted/APLM-Laboratorios.[1]

[1] Foi enviado um convite de colaborador para o utilizador **joaojdacosta**.